Guía de Criterio Arquitectónico

Un error común cuando se programa con IA es creer que porque la aplicación "funciona visualmente", el código está correcto. La IA buscará el camino algorítmico más rápido para darte una respuesta funcional, pero ese camino suele ser el más ineficiente en términos de consumo de recursos.

Saber codear no significa solo saber escribir texto; significa **entender el porqué de cada cosa**. Tu criterio es la única defensa contra el desperdicio masivo de recursos.

1. El Peligro del Desperdicio de Recursos

Si no auditas las decisiones que toma la IA, puedes generar un sistema que colapse cuando tenga más de 100 usuarios.

El Escenario (El error de la IA):
Le pides a la IA: *"Muestra el total de ventas del mes"*. La IA podría generar código en React que haga esto:

1. Pide a la BD **todos** los registros (ej. 50,000 filas).
2. Descarga esos 50,000 registros al navegador del celular.
3. Hace un bucle `for` en JavaScript para sumarlos.
4. Muestra el número.

¡Desastre arquitectónico! Desperdiciaste ancho de banda y cobraste 50,000 llamadas facturables de lectura en tu base de datos.

2. Delegación Correcta de Cargas

Para evitar este desperdicio, debes entender para qué está diseñada cada capa y exigirle a la IA que respete esos roles.

A) La Base de Datos (Supabase / SQL)

Las BD están hiper-optimizadas para cálculos a velocidades absurdas.

- **La regla:** Cualquier cálculo sobre datos masivos (sumas, filtrado) **debe hacerse en la base de datos**.
- **La corrección PCA:** Exígele a la IA: *"No descargues las filas. Créame una función en Supabase que use SUM() y me devuelva únicamente el número final."*

B) El Backend (Edge Functions)

Es tu capa de seguridad y lógica de negocio privada.

- **La regla:** Usa el backend para comunicarse de forma segura con APIs externas o procesar lógica que sea demasiado pesada para un celular pero no nativa de SQL (ej. redimensionar una imagen).

C) El Frontend (React)

El navegador del usuario tiene recursos limitados.

- **La regla:** El Frontend debe ser "Tonto" (*Dumb UI*). Su único trabajo es tomar datos ya procesados (el número final que le dio la BD) y pintarlos en la pantalla.

3. FinOps y la Economía de Tokens

Cuidar la memoria del servidor es vital, pero olvidamos el costo más grande en el desarrollo moderno: **La facturación de la IA**.

Las empresas pierden fortunas pagando *tokens* de APIs porque los desarrolladores novatos envían el código de todo el proyecto en cada mensaje.

El Criterio FinOps: Un ingeniero de élite domina la "Gestión de Contexto por Demanda". Usas resúmenes (tu `AGENTS.md`) en lugar de pasar archivos enteros crudos. Así evitas alucinaciones y demuestras responsabilidad financiera.

4. El Valor de Entender el "Por Qué"

El código ineficiente suele funcionar perfectamente en entornos de prueba locales. Desarrollar tu criterio arquitectónico significa que tu cerebro active una alarma y diga: *"¿Por qué estoy iterando un arreglo gigante en React si las bases de datos tienen funciones nativas para sumar?"*

Reflexión Final

La IA es un obrero increíblemente rápido, pero no tiene consciencia del costo de los servidores o la factura de OpenAI. Ese nivel de criterio, responsabilidad y visión a largo plazo es exactamente lo que te convierte en un Arquitecto de Software.